

FAQ

CS100 HW8 I, Zombie

目录

1	CMake / CMake-GUI 爆红字	2
2	MacOS 上常见问题	2
3	我的游戏运行起来有 bug 但看不出在哪怎么办?	2
4	GameWorld.hpp 中的 <code>std::enable_shared_from_this</code> 是什么?	2
5	[ERROR] Error loading asset ***.png	3
6	我删除 <code>std::list</code> 内元素的时候, 为什么程序会崩溃?	4
7	使用了未定义类型 <code>GameWorld</code> / <code>GameWorld is an incomplete type</code> / <code>undeclared identifiers</code> / <code>not overriding virtual functions</code> / 为什么要把函数的声明和定义分开在 .hpp 和 .cpp 文件里? 都写在 .hpp 里为什么报错? / 循环依赖与 forward declaration	5
8	Visual Studio “Linking error: 无法解析的外部符号”, 或者 GCC <code>ld: undefined reference to xxxx</code>	5
9	另一些 <code>#include</code> 错误: 我没有循环引用, CMakeLists 也改了, 还是找不到被 <code>#include</code> 的文件?	6
10	VS (中文版) 的默认字体好难看, 影响我编程了	7
11	我想把我的游戏打包出来给别人玩, 可以吗?	7
12	Debug 和 Release 是什么? 有什么区别?	7
13	如何添加文字?	8
14	我想做一份自己的植物或僵尸的图片资源显示进游戏, 应该怎么做?	8
15	VSCode 相关问题	8
15.1	生成 build	9
15.2	编译	10
15.3	运行、调试	10
15.4	配置 IntelliSense	10

1 CMake / CMake-GUI 爆红字

如果是 warning 的话，有可能不必担心，看到 build 完成了可以 generate 即可。

如果是 error，下面的几行里就包含报错信息。可以自己检索一下找找原因或者 Piazza 发帖。

2 MacOS 上常见问题

Could Not find X11: X11是一个图形窗口库。试一下 `brew install xquartz --cask` 命令安装 `xquartz`，或者搜索一下相关信息，安装一个 X11。

CMake build 不出来：有往年同学反映过是 `toolchain` 需要用 `gcc` 而不能用 `clang`。先 `brew install gcc` 安装一个 `gcc`，再在根目录的 `CMakeLists.txt` 的第 3 行之前添加：

```
1 set(CMAKE_C_COMPILER "/opt/homebrew/bin/gcc-14")
2 set(CMAKE_CXX_COMPILER "/opt/homebrew/bin/g++-14")
```

因为历年 MacOS 问题五花八门而经验也较少，建议开始之前去看一下 CMake 教学视频。

3 我的游戏运行起来有 bug 但看不出在哪怎么办？

你之前的所有 debug 经验，在这个项目里都可以使用。你可以通过 `std::cout` 向控制台打印一些输出，可以将一些没有贴图的物体换上贴图以便显示，当然也可以使用 `debugger`，在指定代码处打上断点或逐行运行。

VSCode 的 CMake Debug 功能的配置方式请看 VSCode 相关问题。

4 GameWorld.hpp 中的 `std::enable_shared_from_this` 是什么？

省流：这是对 `this` 指针的“智能指针版替代”。当你需要在 `GameWorld` 里获得一个指向自己的 `std::shared_ptr` 时，可以调用 `shared_from_this()`。

一般地，通过让一个类 `X` 继承自 `std::enable_shared_from_this<X>`，我们就能够在 `X` 的内部安全地获得一个管理 `*this` 的 `std::shared_ptr<X>`。注意，调用 `shared_from_this()` 的前提是：当前已经存在某个 `std::shared_ptr` 在管理这个对象。

考虑下面这段错误示例：

```
1 struct X;
2
3 void do_something(std::shared_ptr<X> ptr);
4
5 std::vector<std::shared_ptr<X>> gObjects;
6
7 struct X {
8     void some_action() {
```

```

9     do_something(std::shared_ptr<X>(this)); // 这将导致灾难!
10    // shared_ptr<X>(this) 创建了一个新的 shared_ptr,
11    // 但其在现在 *this 已经在被 gObjects[0] 管理着了。
12    // 这两个 shared_ptr 彼此并不知道对方的存在。
13 }
14 };
15
16 int main() {
17     gObjects.push_back(std::make_shared<X>());
18     gObjects.front()->some_action();
19 }

```

要解决这个问题，需要这样做：

```

1 struct X;
2
3 void do_something(std::shared_ptr<X> ptr);
4
5 std::vector<std::shared_ptr<X>> gObjects;
6
7 struct X : public std::enable_shared_from_this<X> {
8     void some_action() {
9         do_something(shared_from_this());
10    }
11 };
12
13 int main() {
14     gObjects.push_back(std::make_shared<X>());
15     gObjects.front()->some_action();
16 }

```

注意：

1. 并非需要将所有的 `this` 都替换为 `shared_from_this()`。不要胡乱使用它。
2. 调用 `shared_from_this()` 的前提是当前 `*this` 正被某个 `shared_ptr` 管理着。否则它会抛出 `std::bad_weak_ptr` 异常。

5 [ERROR] Error loading asset ***.png

ERROR 这是程序没有找到对应路径的图片素材产生的报错。

以下内容主要以 Windows 上的情况举例，但实际上在各平台都是相通的。

我们来尝试理解这个报错。假如它说的是 `Error loading asset ../assets/background.png`，意思就是它试图寻找并加载 `../assets/background.png` 但失败了。这是一个相对路径：`..` 的意思是相对于“当前位置”的上一级目录，也就是说它要找的东西是相对于“当前位置”的上一级目录中的 `assets` 文件夹下的 `background.png`。

那么问题就来了，“当前位置”是什么？注意，“当前位置”不是 `hw8` 文件夹，不是 `main.cpp` 的位置，也不是生成的 `PvZ` 可执行文件的位置。不管你用何种方式运行这个程序，它实际上都是从一个终端启动的，**Visual Studio** 会弹出一个终端窗口，而 **VSCode** 或者 **CLion** 会使用自己的终端面板。这个终端的当前工作目录 (current working directory, `cwd`) 才是我们要找的“当前位置”。

你现在就可以找到运行这个程序的终端，看它的 `cwd` 究竟在哪。一般来说终端会将自己的 `cwd` 作为提示语写出来，比如：

```
1 PS D:\CS100\hw\hw8\build\bin>
```

这意思就是 `cwd` 为 `D:/CS100/hw/hw8/build/bin`。一个热知识：尽管在 Windows 上路径分隔符是 `\`，但 **Powershell**、C/C++ 的 `#include`、**CMake** 等绝大多数工具都是允许用 `/` 的，所以我们也就不再区分了。假如 `assets` 是和 `build` 并列的文件夹，那么在当前位置——也就是 `bin` 下——当然是找不到 `../assets/` 的，因为正确的相对路径是 `../../assets/`。

现在你可以试一下 `cd ..`，即将工作目录切换到上一级目录，也就是 `build`，效果应该是这样：

```
1 PS D:\CS100\hw\hw8\build\bin> cd ..
2 PS D:\CS100\hw\hw8\build>
```

输入你的可执行文件相对于当前位置的相对路径来运行它。比如，假如可执行文件 `PvZ.exe` 位于 `D:/CS100/hw/hw8/build/bin/Debug/PvZ.exe` (**Visual Studio** 通常会把它放在这儿)，那就输入 `bin/Debug/PvZ` (`.exe` 可写可不写)。现在它应该就能正常加载 `assets` 了。

搞明白了这个原理，那么就有了基本的解决问题的思路。

一种办法是改 `cwd`。假如你运行这个程序的方式是在 **VSCode** 中启动“CMake: Debug”或者“CMake: Run Without Debugging”，你就要想办法对 **VSCode** 的 **CMake** 相关插件在运行程序时的 `cwd` 进行修改，这一点请看 **VSCode** 相关问题：运行、调试一节。假如你使用的是什么别的软件，那么肯定也是有相应的配置的，请自行 STFW (Search The Friendly Web)。当然，我 (GKxx) 自己一直都是直接在 **VSCode** 的终端输入可执行文件的相对路径来运行程序的，我会通过设置 `"terminal.integrated.cwd": "build"` 来使得每次我按 `Ctrl + `` 启动终端的时候都会默认切换到 `build`，这也不失为一种办法。

另一种办法就是改 `utils.hpp` 中的 `ASSET_DIR`，比如多加一层 `../`，或者干脆把它改成绝对路径。注意：尽管将它改成绝对路径是非常简单粗暴的解决方法，但这并不意味着以上有关 `cwd` 的解释你可以不用掌握，它们实际上就是终端使用的基本常识。

6 我删除 `std::list` 内元素的时候，为什么程序会崩溃？

省流：对于 `std::list`，一边遍历一边删除指定元素时，正确方法是使用迭代器作为循环变量并调用 `erase()`，而不是使用 `range-for`。

`range-for` 本质上也是在使用迭代器遍历你的 `list`。当你删除 `list` 中元素时，会无效化 (invalidate) 指向被删除元素的迭代器，也就是 `range-for` 在背后使用的那个，于是就会引发错误。

这其实很好理解：所谓“链表”，就是在每个元素身上记录它前一个和后一个元素的地址。如果你把 `iter` 指向的元素删除了，`++iter` 还怎么执行呢？它怎么知道自己指向谁、下一个又是谁呢？

当然，你也可以考虑使用 `std::list<T>::remove_if`。文档原作者也明确说过，这通常是更推荐的方案。

7 使用了未定义类型 `GameWorld` / `GameWorld is an incomplete type / undeclared identifiers / not overriding virtual functions / 为什么要把函数的声明和定义分开在 .hpp 和 .cpp 文件里？都写在 .hpp 里为什么报错？ / 循环依赖与 forward declaration`

省流：`#include` 本质上就是简单的文本复制粘贴。两个头文件互相 `#include` 很容易制造循环依赖；把声明和定义分开写在 `.hpp` 和 `.cpp` 中，很多时候就是为了解决这个问题。

假如 `A.hpp` 和 `B.hpp` 中分别定义了类 `A` 和类 `B`。当它们互相 `#include` 时，若 `A` 和 `B` 又都需要对方的完整定义，就一定会会有一个类因为“出现在前面”而不满足依赖。

不过这种情况一般不会真的走到“两边都必须完整定义”那一步。更常见的是：其中一方只需要另一方的指针或引用（包括智能指针和容器等）。这时其实不需要完整定义，只需要一个前向声明，例如：

```
1 class A;
```

这条声明只告诉编译器“`A` 是一个类”，但不会告诉它 `A` 的成员、大小、方法实现等信息。所以：

- 如果你只是存一个指针或引用，前向声明通常就够了。
- 如果你要调用 `A` 的成员函数、创建 `A` 对象、访问其具体定义，就仍然必须 `#include "A.hpp"`。

这也是为什么很多依赖“完整定义”的代码应该放进 `.cpp` 文件里：`.cpp` 一般不会再被别的文件 `#include`，因此更不容易形成循环依赖。

8 Visual Studio “Linking error: 无法解析的外部符号”，或者 GCC `ld: undefined reference to xxxx`

这个错误是链接器的报错，通常意味着你调用了一个“有声明、没定义”或者“有定义、但没被链接进来”的函数。常见情况大致有两种：

1. 某个被使用的函数确实只有声明、没有定义。特别地，非纯虚函数也必须有定义，否则会影

响运行时信息的生成。

2. 某个函数虽然定义了，但编译器 / 链接器没有看到它。比如：

- 你把函数写在 `a.cpp` 里，但没有把 `a.cpp` 加进任何 `target`。
- 你已经把实现编译成另一个库了，但当前 `target` 没有在 `target_link_libraries` 里链接它。

VS 的报错可能长这样：

```
1 [build] XXX.lib(XXX.obj): error LNK2019: 无法解析的外部符号 "YYY" ...
```

MinGW / GCC 的报错可能长这样：

```
1 [build] path/to/ld.exe: lib/libXXX.a(XXX.cpp.obj): in function `ZZZ':
2 [build] path/to/SomeSourceFile.cpp:LLL:(.text+AAA): undefined reference to `YYY'
```

如果确认 `YYY` 的定义在另一个 `target` 里，那就需要把它链接进来，例如：

```
1 target_link_libraries(
2     XXX
3     Library1
4     Library2
5     # ...
6     TargetContainingYYY
7 )
```

9 另一些 `#include` 错误：我没有循环引用，`CMakeLists` 也改了，还是找不到被 `#include` 的文件？

假设有如下文件结构：

```
1 .
2 |— A/
3 |   |— A.hpp
4 |   |— A.cpp
5 |   |— CMakeLists.txt
6 |— B/
7 |   |— B.hpp
8 |   |— B.cpp
9 |   |— CMakeLists.txt
10 |— C/
11 |   |— C.hpp
12 |   |— C.cpp
13 |   |— CMakeLists.txt
```

假如 `A.cpp` 中需要 `#include "B.hpp"`，很多同学会直接把 `B` 的路径加到 `A/CMakeLists.txt`

的 `target_include_directories` 里。再假如 `B.hpp` 里又写了 `#include "C.hpp"`，你又把 `C` 的路径加到 `B/CMakeLists.txt` 里。

问题在于：`#include` 本质上就是文本复制粘贴。当编译器把 `B.hpp` 的内容粘到 `A.cpp` 里时，`#include "C.hpp"` 也一起被带进了 `A.cpp`。但你并没有在 `A` 自己的编译环境里告诉编译器去哪找 `C.hpp`，于是它还是会报找不到。

这也是为什么只靠给各个 `target_include_directories` 打补丁，往往会把工程维护成“打地鼠”状态：你刚修好一个地方，另一个看似无关的 `#include` 又会引发新问题。

最简单的做法就是统一约定：所有 `#include` 都从 `pvz/` 开始写，并且在根 `CMakeLists.txt` 里统一设置：

```
1 include_directories(${CMAKE_SOURCE_DIR}/include)
```

10 VS（中文版）的默认字体好难看，影响我编程了

确实难看并且不适合作为代码字体。工具-选项-环境-字体和颜色，将“文本编辑器”的字体设置为适合编程的等宽字体，例如系统内置的 `Consolas` 或 `Cascadia Mono` 等。

也可以现在卸载 Windows，使用 Linux。

11 我想把我的游戏打包出来给别人玩，可以吗？

当然可以，你需要做的是像示例程序一样，将程序的可执行文件，必要的库（Windows dll）和图片素材文件夹整理出来。

你可以在 `build/Debug` 或 `build/Release` 中找到你的可执行文件，在 Windows 下，你也可能会找到一些 dll 库。这些文件都是必要的。

要注意的是，你定义的 `ASSET_DIR` 如果是相对路径的话，在你双击 `.exe` 运行游戏时，它应该从到 `.exe` 可执行文件所在位置到 `Assets` 目录的相对路径。（注意与上一条的区别，在 `project` 里运行时会被解析为从项目工程文件开始的路径）

12 Debug 和 Release 是什么？有什么区别？

你的项目能以 **Debug/Release** 两种方式生成 (**build**)。Debug 模式下可以接入调试器 (**debugger**)，利用断点或堆栈检查来调试代码，而 Release 模式则会进行一些编译器优化等，获得更快的运行速度。并且，Release 模式下编译时会定义 `NDEBUG` 宏，这会使得你程序里的所有 `assert` 都被去除。

你的项目的默认生成模式一般是 **Debug**。你可以在你的开发环境 (**IDE**) 的顶端状态栏看到并调整生成模式。其实 **CMake** 提供了 `CMAKE_BUILD_TYPE` 这个变量，在最初 **build** 项目时设定生成模式，但是这个设定一般会被 **IDE** 接管覆盖。

13 如何添加文字?

在 Framework 目录下有 `TextBase.hpp` 文件。其构造函数中，必须提供的参数是 `x` 和 `y`。其初始文字内容默认为空字符串、颜色默认为 0, 0, 0（黑色），居中模式默认为 `true`。

你可以通过 `TextBase.hpp` 中提供的方法来修改它的位置、内容、颜色。

当你想要创建一个文字时，我们建议你以对待 **GameObject** 类似的方式创建它，即使用指针去动态地分配。与 **GameObject** 不同的是，你不必将所有文字都存放在 **GameWorld** 里的某个容器中，因为游戏对象每帧都需要 `Update()` 做出不同的行为，而文字不需要。通过将文字与一些 **GameObject** 绑定，你还可以做到比如显示每棵植物的 HP 或冷却时间等功能。

14 我想做一份自己的植物或僵尸的图片资源显示进游戏，应该怎么做?

我们欢迎你在有余力的情况下这样做，因为视觉效果的不同是游戏对玩家的很重要的反馈。

想要获得新的图片资源，对游戏中现有的图片素材进行 PS /调色是最简单的方法。将新的资源放进 `assets` 文件夹后，你还需要：

- 在 `utils.hpp` 里新建一个属于你的 `IMGID`；如果你加入了新的动画，也可以创建新的 `ANIMID`。
- 在 `Framework/SpriteManager.cpp` 里加载你的新素材。15-50 行中的代码用来完成这些，并且 `hard-code` 录入了素材的大小和帧数信息。你可以复制一行，参考它对应的图片素材与它的写法，填入你的信息。每个参数的意义是：

```

1 {
2   EncodeAnim(/* 你的 IMGID */, /* 你的 ANIMID */),
3   SpriteInfo{
4     /* 文件名 */,
5     /* 图片总宽 */, /* 图片总高 */,
6     /* 每帧宽 */, /* 每帧高 */,
7     /* 每行帧数，默认 1 */,
8     /* 动画长度，默认 1 */
9   }
10 }
```

【注意】 你加入的上面这行中，至少有两处需要更改，分别是 `EncodeAnim` 的 `IMGID` 和 `SpriteInfo` 的文件名。

15 VSCode 相关问题

在开始之前，请确保当前工作区的绝对路径上没有任何非 ASCII 字符、空格、可能引发问题的标点符号。像给文件夹命名为“作业”、“大一下”这样的中文名的习惯建议趁早改，因为不支持中文路径的软件非常多。

要使用 **VSCode** 配合 **CMake** 进行项目构建, 首先需要安装 **CMake**、**CMake Tools** 和 **CMake Language Support** 这三个扩展 (“扩展” 又名 “插件”)。

你还需要了解一个非常基本的知识: 对于当前工作区 (即, 现在打开的最外层文件夹) 的配置, 是通过在当前工作区下的 `.vscode` 文件夹中的若干配置文件来完成的。这个 `.vscode` 文件夹必须直接位于当前工作区, 不能在某个更里面的文件夹中。一般来说, `settings.json` 包含了绝大多数的设置项, `tasks.json` 通常负责项目的编译, `launch.json` 通常负责程序的运行, 而某些扩展可能会约定属于它自己的配置文件, 例如 `c_cpp_properties.json` 是 C/C++ 这个扩展自己的特殊配置文件。在本学期初的搭环境视频中, 我们其实已经解释了这些配置文件的一些细节, 可能那时对代码十分陌生的你还未能很好地理解它们, 但是经过一个学期的学习之后, 你应该逐渐认识它们并学会编写它们了。

接下来要明确的是, 我们需要解决以下问题:

1. 使用 **CMake** 生成 **build system** 所需的文件, 通俗点说就是生成 `build` 文件夹以及里面的一大堆东西。
2. 调用 **build system** 来根据 `build` 文件夹里的内容对项目进行编译。
3. 编译成功后, 运行、调试可执行文件。
4. 配置 **IntelliSense**, 使得你写代码时可以获得补全、查找、重命名、实时报错等功能。

一个热知识: **Ctrl + Shift + p** (在 Mac 上是 **Command + Shift + p**) 是 **VSCode** 的 “万能按键”, 在这里你可以输入一些内容, **VSCode** 会帮你找到最匹配的配置或功能。

15.1 生成 build

按 **Ctrl + Shift + p** 输入 **cmake configure**, 最优匹配应当是 “CMake: Configure”, 回车。接下来如果让你选 Kit (第一次执行通常需要), Windows 上应该可以选 **Visual Studio 17 2022 amd64** 或者最高版本的 **gcc**, Mac 上选最高版本的 **gcc**。特别是对于 Windows, 如果这个选项不存在, 有可能是没有正确安装 Visual Studio, 请先去安装最新版的 **Visual Studio**; 如果已经装好了, 可以选 **[Scan for kits]** 刷新一下, 应该就有了。

默认情况下, **CMake** 插件会将当前工作区中 (最外层文件夹) 的 `CMakeLists.txt` 作为根。如果这个文件不存在, 它会报错。修改这一行为的方式是在 `settings.json` 中加入一项 `"cmake.sourceDirectory"`, 例如

```
1 "cmake.sourceDirectory": "${workspaceFolder}/attachments"
```

就会让 **CMake** 插件去当前工作区下的 `attachments` 文件夹中寻找 `CMakeLists.txt` 作为根。这里我们使用了一个变量 `${workspaceFolder}`, 这也是在 **VSCode** 配置中十分常见的。

在生成的过程中你需要密切关注的, 是它采用的 `generator` 是什么, 特别是 **Windows** 上可能会有问题。在你启动 “CMake: Configure” 时, **Output** 面板中会显示一些日志, 其中最前面应该有一段 `[proc] Executing command:` 开头的内容, 它表示究竟执行了什么指令。找到 `-G` 后面紧跟

着的东西，它就是现在 **CMake** 在使用的 **generator**。

对于 Windows，这一项应该是 Visual Studio 17 2022。如果是 MinGW Makefiles，则在本次作业中大概率会引发错误（具体原因未知）。你也可以通过生成的 build 文件夹中的内容来看：Visual Studio 对应的是某个 .sln 文件，而 **MinGW Makefiles** 对应的是一个名为 Makefile（无后缀名）的文件。可以在 settings.json 中设置：

```
1 "cmake.generator": "MinGW Makefiles"
```

如果之前已经生成了 build，换了一个 **generator** 之后要重新生成，你需要把 build 文件夹删除。

15.2 编译

按 **Ctrl + Shift + p** 输入 **cmake build**，最优匹配应当是“CMake: Build”，回车。如果之前没有启动过“CMake: Configure”，“CMake: Build”会启动“CMake Configure”生成 build，在此过程中遇到问题请看上一节。

编译的过程中，如果在 **Output** 面板中显示的中文有乱码，去全局设置（按 **Ctrl + 逗号**）找“Cmake: Output Log Encoding”，将其值改为 **UTF-8**。

编译完成后，可以留意一下生成的可执行文件的位置，它可能在 build/bin、build/bin/Debug 或者什么类似的目录里。

15.3 运行、调试

编译成功后，按 **Ctrl + Shift + p** 输入 **cmake run without debug**，最优匹配应当是“CMake: Run Without Debugging”，回车，就是非调试地运行。如果输入 **cmake debug**，最优匹配应当是“CMake: Debug”，就是调试。

无论是调试还是非调试运行，我们可能都需要将运行时的工作目录设为 build，以正确加载 assets。这一项的配置是 "cmake.debugConfig" 的 "cwd" 子项：

```
1 "cmake.debugConfig": {  
2   "cwd": "${workspaceFolder}/build"  
3 }
```

其它的事情跟正常 debug 没啥区别，你仍然可以打断点、看 local variables 和 call stack 等。

15.4 配置 IntelliSense

一般情况下，C/C++ 的 **IntelliSense** 是由插件“C/C++”负责的，你可以在当前工作区的 .vscode 中编写 c_cpp_properties.json 来对其进行配置。

现在我们有 **CMake** 了，影响是什么呢？是我们的编译方式变复杂了，而且编译指令是由 **CMake** 生成的相关文件给出的，想要直接获得编译指令并翻译为 c_cpp_properties.json 的配置项

是非常困难的。幸好，有一个简单的办法，那就是让 **CMake** 相关插件直接和 C/C++ 插件沟通：

```
1 // c_cpp_properties.json 的内容
2 {
3   "configurations": [
4     {
5       "name": "CMake",
6       "configurationProvider": "ms-vscode.cmake-tools",
7     }
8   ],
9   "version": 4
10 }
```

这里，"name" 一项并不重要，它可以是任何值。关键之处是 "configurationProvider" 一项，设为 "ms-vscode.cmake-tools" 就是让 CMake Tools 插件告诉 C/C++ 插件应该怎样配置。就这样。

现在你可以试一下：比如随便删掉代码里的一个分号，看看 **VSCode** 有没有给你画红线；或者按 Ctrl 左击某个 #include 的文件，看看是否能正常跳转。